

23L620 – INNOVATION PRACTICES

DIGITAL IMPLEMENTATION OF BEAMFORMING TECHNIQUE USING CORDIC ALGORITHM

BUVNAPATHI K (Roll no. 23L105)
SANKARANARAYANAN A (Roll no. 23L148)
NAVEEN M (Roll no. 24L404)

Dissertation submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING

BRANCH: ELECTRONICS AND COMMUNICATION ENGINEERING

of Anna University



March 2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

**DIGITAL IMPLEMENTATION OF BEAMFORMING
TECHNIQUE USING CORDIC ALGORITHM**

Bona fide record of work done by

BUVNAPATHI K (Roll no. 23L105)

SANKARANARAYANAN A (Roll no. 23L148)

NAVEEN M (Roll no. 24L404)

Dissertation submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING

Branch: ELECTRONICS AND COMMUNICATION ENGINEERING
of Anna University

March, 2026

.....
Dr G. Umamaheswari

Faculty Guide

.....
Dr V. Krishnaveni

Head of the Department

Dr A. Praveena
ACD Communications Pvt Ltd. Hyderabad
Industry mentor

Certified that the candidate was examined in the viva-voce examination held on

.....

.....
(Internal Examiner)

.....
(External Examiner)

CONTENTS

CHAPTER	Page. No
Acknowledgement	(v)
Synopsis.....	(vi)
List of Tables.....	(vii)
List of Figures.....	(viii)
1. INTRODUCTION.....	1
1.1 Need for the project	1
1.2 Introduction of Concepts	1
1.3 Problem Statement	1
1.4 Objectives	1
1.5 Overview of the project	2
1.6 Report Organisation	3
2. LITERATURE SURVEY.....	4
3. BEAMFORMING ARCHITECTURE AND METHODOLOGY.....	5
3.1 Beamforming architecture	5
3.2 Software Flow Diagram	7
3.3 Hardware Connection Diagram	8
3.4 Working Principle	8
3.5 Novelty	9
4. RESULT AND PERFORMANCE ANALYSIS.....	10
4.1 Simulink Model Verification (Software Simulation)	10
4.2 Hardware Synthesis and Schematic Design	13
4.3 Hardware Validation (Time Domain)	14
4.4 Spatial Filtering and Radiation Pattern	16
4.5 Hardware Utilisation and Timing Analysis	17

4.6 Power Consumption Analysis	18
5. CONCLUSION AND FUTURE SCOPE	20
BLIOGRAPHY.....	22
APPENDICES	23
Appendix 1: Development Tools and Software Environment	23
Appendix 2: Hardware Platform Specifications	24
Appendix 3: Core Algorithms and Protocols	24
Appendix 4: Python Source Code for System Validation	26

ACKNOWLEDGEMENT

We express our sincere thanks to the Principal, Dr K. Prakasan, for his encouragement and unwavering support, which motivated us to push the boundaries of our work. Our gratitude further extends to our Head of Department, Dr V. Krishnaveni, and our Program Coordinator, Dr S. Hemachitra, for providing the platform and opportunity to undertake this challenging and innovative project. Their strong leadership and commitment to academic excellence have been truly inspiring.

We are grateful to our Project Guide, Dr G. Umamaheswari, for providing a clear vision for the system. Their invaluable insights, constant encouragement, and regular critical reviews have been instrumental in guiding us throughout this work and shaping the project to its current form.

We would also like to express our sincere gratitude to our industry mentor, Dr A Praveena (ACD Communications Pvt. Ltd), for his valuable guidance, practical insights, and support.

We would like to thank our tutor, Dr A. Kannammal, for his thoughtful guidance and constructive feedback, which greatly enriched our learning experience.

Our gratitude extends to the entire staff of the Electronics and Communication Department, whose constant encouragement, helpful suggestions, and moral support played a crucial role in keeping us motivated throughout the duration of our project.

Finally, we acknowledge everyone who has been associated with our project at any stage, but whose names may not find a place in this acknowledgement. Your support, directly or indirectly, has been invaluable, and we are deeply indebted to you all.

SYNOPSIS

Modern digital communication, radar, and acoustic systems rely heavily on beamforming techniques to isolate targeted signals in crowded, interference-heavy environments. However, digital beamforming requires continuous, high-speed spatial filtering, which involves computing vast arrays of complex mathematical operations. When these parallelizable, math-intensive tasks—such as complex multiply-accumulate operations—are executed purely in software on standard microprocessors or digital signal processors, they create a severe processing bottleneck. Software must execute these phase rotations sequentially, leading to unacceptable latency and excessive dynamic power consumption. Consequently, there is a critical need for low-power, edge-capable hardware accelerators that can offload these mathematical burdens into dedicated silicon, achieving real-time parallel execution without requiring the massive overhead of premium, high-cost RF-SoC platforms.

To address this processing bottleneck, this project explores the design, implementation, and hardware acceleration of a digital beamforming system utilising a strict hardware-software co-design approach. The proposed architecture successfully implements a 4-element Uniform Linear Array (ULA) configuration mapped onto a System-on-Chip (SoC) environment. By partitioning the system, the architecture designates the Processing System to handle high-level control, testbench execution, and continuous wavefront synthesis. Simultaneously, the core complex phase-rotation and signal accumulation mathematics are offloaded to a custom Intellectual Property block residing in the Programmable Logic. This allows the system to process all four antenna feeds simultaneously in a single clock cycle, dramatically eliminating the sequential software delays and validating the feasibility of fixed-point hardware accelerators for array processing.

The development pipeline integrates a suite of advanced engineering tools, beginning with MathWorks Simulink for Model-Based Design, to theoretically validate the beamforming algorithms. HDL Coder is subsequently utilised to automatically translate these continuous mathematical models into synthesizable Verilog IP. AMD Xilinx Vivado handles the hardware synthesis, block design, and final SoC integration, targeting the Zynq-7000 silicon on the PYNQ-Z2 development board. The expected results of this project include the successful deployment of a Hardware-in-the-Loop testbed capable of achieving 100 Mega-Samples Per Second throughput. The hardware is expected to mathematically prove near-perfect signal alignment and spatial nulling with minimal quantisation error, all while operating at a total on-chip power consumption of under 1.6 Watts.

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of literature surveyed for FPGA-based digital beamforming and system partitioning.	4
4.1	AXI4-Lite Memory-Mapped Register Allocation	14

LIST OF FIGURES

Fig.No.	Title	Page No.
1.1	Concise block diagram	2
3.1	Top-level block diagram of the system architecture illustrating the hardware-software partitioning between the Processing System (PS) and Programmable Logic (PL).	5
3.2	Internal block diagram of the Custom Beamformer IP Core, detailing the AXI register interface, CORDIC phase calculation, and the parallel adder tree.	6
3.3	Hardware-Software Co-Design Data Flow and Execution Sequence	7
3.4	Physical hardware connection topology illustrating the power and network interfaces between the host laptop and the PYNQ-Z2 development board.	8
4.1	Simulink top-level block design illustrating the four discrete antenna input channels and the target steering angle (θ) feeding into the core beamforming IP subsystem.	10
4.2	Internal schematic of the beamforming subsystem detailing the CORDIC-based complex weight generation, parallel mathematical multiplication, and the adder tree accumulation pipeline.	11
4.3	Simulink Scope output for the Steering Matched Condition, demonstrating perfect constructive interference.	12
4.4	Simulink Scope output for the Steering Mismatch Condition, demonstrating partial phase cancellation and suppressed signal output.	12
4.5	Overall, Vivado Block Design featuring the Zynq Processing System, AXI Interconnect, and Custom Beamformer IP.	13
4.6	Hardware execution of discrete time-domain signals. Dashed lines indicate the out-of-phase antenna inputs; the solid line indicates the combined hardware output.	15
4.7	Hardware-generated radiation pattern demonstrating the main lobe and spatial nulls.	16
4.8	Post-implementation Resource Utilisation Report.	17
4.9	Vivado Timing Summary demonstrating positive slack.	18
4.10	Vivado Implementation Power Report detailing dynamic and static power consumption.	18

CHAPTER 1

INTRODUCTION

This chapter introduces the fundamental concepts of digital beamforming and establishes the critical need for a hardware-accelerated solution to overcome the latency and power limitations of traditional software-based systems. It defines the primary problem statement and the specific objectives of the proposed research. Finally, it provides a high-level overview of the hardware-software co-design approach using the Zynq-7000 SoC and outlines the organisational structure of the subsequent chapters.

1.1 Need for the Project

In dense wireless environments, signals from multiple users and interferers overlap, degrading receiver performance. To overcome these limitations, directional reception techniques are required to selectively enhance signals from a desired direction. However, software-based beamforming suffers from unacceptable latency and excessive power consumption. Therefore, there is a critical need for an efficient hardware accelerator to perform these parallel computations and achieve real-time directional reception.

1.2 Introduction of Concepts

Digital beamforming is an advanced signal processing technique that achieves spatial selectivity by combining inputs from multiple sensor elements. By mathematically manipulating the signals across a phased array, the system creates constructive interference for signals arriving from a targeted angle (the main lobe) and destructive interference for signals from other directions (the nulls).

At the core of a uniform linear array is the concept of phase delay. When a wavefront arrives at an array from a specific spatial angle, it reaches each individual element at a slightly different time. To digitally align these signals, the system must apply calculated complex weights—specifically, real and imaginary phase shifts—to the data stream of each element before summing them together.

Executing these parallelizable, math-intensive tasks purely in software introduces sequential processing delays and high power consumption. Hardware acceleration solves this by mapping the algorithm directly into silicon. This allows all four antenna feeds to be processed simultaneously in a single clock cycle, drastically reducing latency and dynamic power usage while freeing up the main processor for other tasks.

1.3 Problem Statement

Executing complex beamforming mathematics purely in software creates severe latency and excessive power consumption bottlenecks. There is a critical need for a low-

power, cost-effective hardware accelerator to offload these parallel tasks without relying on expensive RF-SoC platforms.

1.4 Objectives

- To design a digital beamforming architecture optimised for hardware mapping and scalability.
- To replace multiplier-intensive operations in digital beamforming with a shift-and-add- based computational approach.
- To reduce power consumption and FPGA resource utilisation compared to conventional architectures.

1.5 Overview of the Project

This project details the end-to-end development of a real-time digital beamformer, bridging high-level algorithmic modelling with low-level silicon deployment. The core mathematical models and phase-shifting logic are developed and verified using Simulink. To bridge the gap to hardware, HDL Coder is utilised to automatically translate the block diagrams into synthesizable Verilog Intellectual Property cores, which are then integrated into a larger hardware design using AMD Xilinx Vivado.

The target hardware is the PYNQ-Z2 development board, which features a Zynq-7000 SoC. This architecture allows for a robust hardware-software partition. The Programmable Logic fabric hosts the custom 4-element beamforming IP, handling the high-speed, parallel complex multiplications and additions required for combining the array inputs. Simultaneously, the Processing System utilises its dual-core ARM processor to run a Linux environment and a Python stack. It interfaces with the Programmable Logic via the AXI-Lite memory-mapped bus to dynamically inject simulated signal data, control steering angles, and retrieve computed output magnitudes in real time.

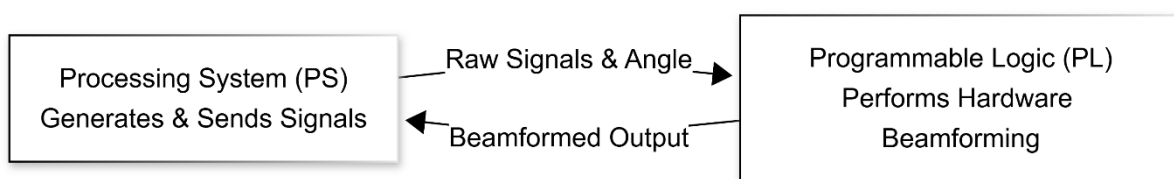


Fig 1.1 Concise block diagram

The project encompasses the successful deployment of this 4-element array, complete with rigorous validation of radiation patterns and hardware power efficiency. (Ref Fig 1.1).

1.6 Report Organisation

The remainder of this dissertation is logically structured to guide the reader through the theoretical foundation, architectural design, physical implementation, and quantitative analysis of the proposed beamforming system.

Chapter 2 presents a comprehensive review of foundational research and contemporary methodologies. It examines existing literature on FPGA-based phase-shift beamformers, hardware-software partitioning within System-on-Chip architectures, Model-Based Design workflows, and automated HDL code generation techniques.

Chapter 3 details the core hardware-software co-design strategy utilized for the testbed. This chapter breaks down the overall system block diagram, explicitly defining the partitioned roles of the Processing System and the Programmable Logic. It also outlines the sequential software flow, internal hardware connections, mathematical working principles, and the novelty of the proposed implementation.

Chapter 4 provides a rigorous, data-driven evaluation of the implemented system. It walks through the initial mathematical verification in Simulink, followed by the physical Hardware-in-the-Loop time-domain validation and spatial radiation pattern generation. The chapter concludes with an exhaustive analysis of the system's hardware resource utilisation, timing margins, and power efficiency as reported by Vivado.

Chapter 5 summarises the consolidated findings of the project, highlighting the successful deployment of the low-power hardware accelerator. It discusses the current limitations of the testbed, proposes ideal real-world applications, and outlines potential avenues for future scaling and adaptive algorithm integration.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews foundational literature on FPGA-based beamforming, System-on-Chip partitioning, and Model-Based Design workflows. By analysing these contemporary methodologies, it establishes the theoretical justification for the proposed hardware accelerator.

Table 2.1: Summary of literature surveyed for FPGA-based digital beamforming and system partitioning.

S. No	Reference	Core Focus/ Methodology	Inference
1	P. Jeyakumar, K. Sudhakar, P. Thanapal, C. Navaneethan, and S. Meenatchi, "FPGA implementation of an efficient phase shift beamformer for narrow band sub-GHz applications," <i>Analog Integrated Circuits and Signal Processing</i> , vol. 122, no. 13, Jan. 2025.	FPGA & CORDIC: Hardware implementation of a Uniform Linear Array (ULA) utilising the CORDIC algorithm for phase shifting.	Validates that offloading Multiply-Accumulate (MAC) and CORDIC math to an FPGA eliminates sequential software delays, achieving single-cycle parallel execution.
2	S. Pulipati <i>et al.</i> , "Xilinx RF-SOC-based Digital Multi-Beam Array Processors for 28/60 GHz Wireless Testbeds," in <i>Moratuwa Engineering Research Conference (MERCon)</i> , Moratuwa, Sri Lanka, 2020, pp. 1-6.	SoC Co-Design: Architectural partitioning between the Processing System (PS) and Programmable Logic (PL) using the AXI bus.	Defends the project's architecture: PS manages high-level control and environment synthesis, while PL executes the high-speed data plane without stalling.
3	T. Khan, "Modelling and Simulating Large Phased Array Systems," presented at <i>MATLAB EXPO</i> , 2017.	Model-Based Design (MBD): Pre-hardware simulation of array geometries and phase delays using visual dataflow environments (Simulink).	Establishes the necessity of theoretically verifying constructive and destructive interference logic before generating any synthesizable hardware code.
4	MathWorks, "HDL Coder Documentation: Generate HDL Code for FPGA and ASIC Designs," MathWorks.com, 2024. [Online].	HDL Translation Pipeline: Fixed-point quantisation, automated Verilog IP generation, and bit-true co-simulation.	Provides the strict engineering pipeline used to convert continuous floating-point models into discrete sfix16_En12 hardware logic and map AXI-Lite registers.

CHAPTER 3

BEAMFORMING ARCHITECTURE AND METHODOLOGY

This chapter details the hardware-software co-design strategy used to construct the beamforming architecture on the Zynq-7000 SoC. It strictly outlines the system partitioning, sequential software execution flow, mathematical working principles, and physical testbed setup.

3.1 Beamforming architecture

The proposed architecture utilises a strict hardware-software co-design partitioning strategy within the Zynq-7000 environment. The overarching system architecture is divided into two primary domains connected via an interconnect bus, as illustrated in the top-level block diagram (refer Fig. 3.1).

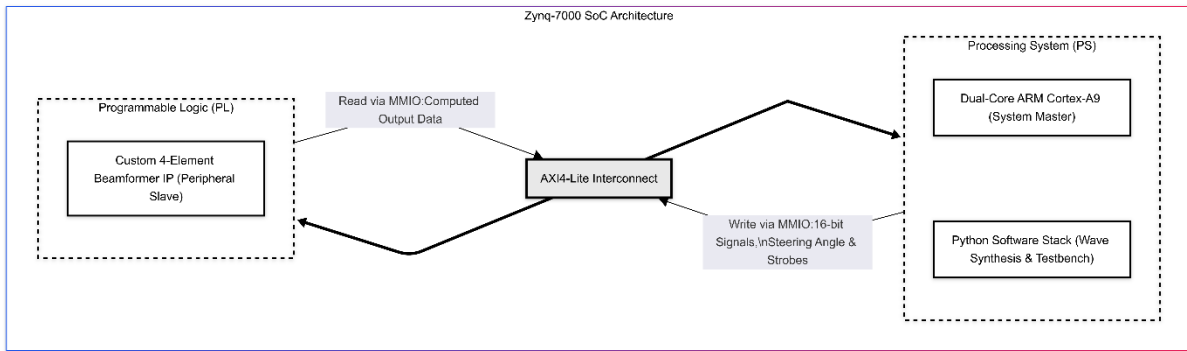


Fig 3.1: Top-level block diagram of the system architecture illustrating the hardware-software partitioning between the Processing System (PS) and Programmable Logic (PL).

The high-level execution flow relies on three core components:

- The Processing System (PS): The dual-core ARM Cortex-A9 processor serves as the system master. It runs the software stack responsible for calculating raw simulated wave geometries, managing user inputs (such as steering angle control), and executing the Python-based environment simulation and testbench.
- The Programmable Logic (PL): The custom 4-element beamformer Intellectual Property (IP), generated via HDL Coder from a Simulink model, resides in the FPGA fabric. It acts as a high-speed peripheral slave utilized for executing parallel mathematical operations.
- The AXI4-Lite Interconnect: The PS and PL are unified by the Advanced eXtensible Interface (AXI). The ARM processor utilises Memory-Mapped I/O (MMIO) to write 16-bit input signals and steering angles to specific physical memory addresses, while hardware "strobe" registers act as execution triggers to coordinate data handoffs.

To further detail the specific mathematical operations occurring within the FPGA fabric, the internal architecture of the custom IP is broken down below (refer Fig3.2).

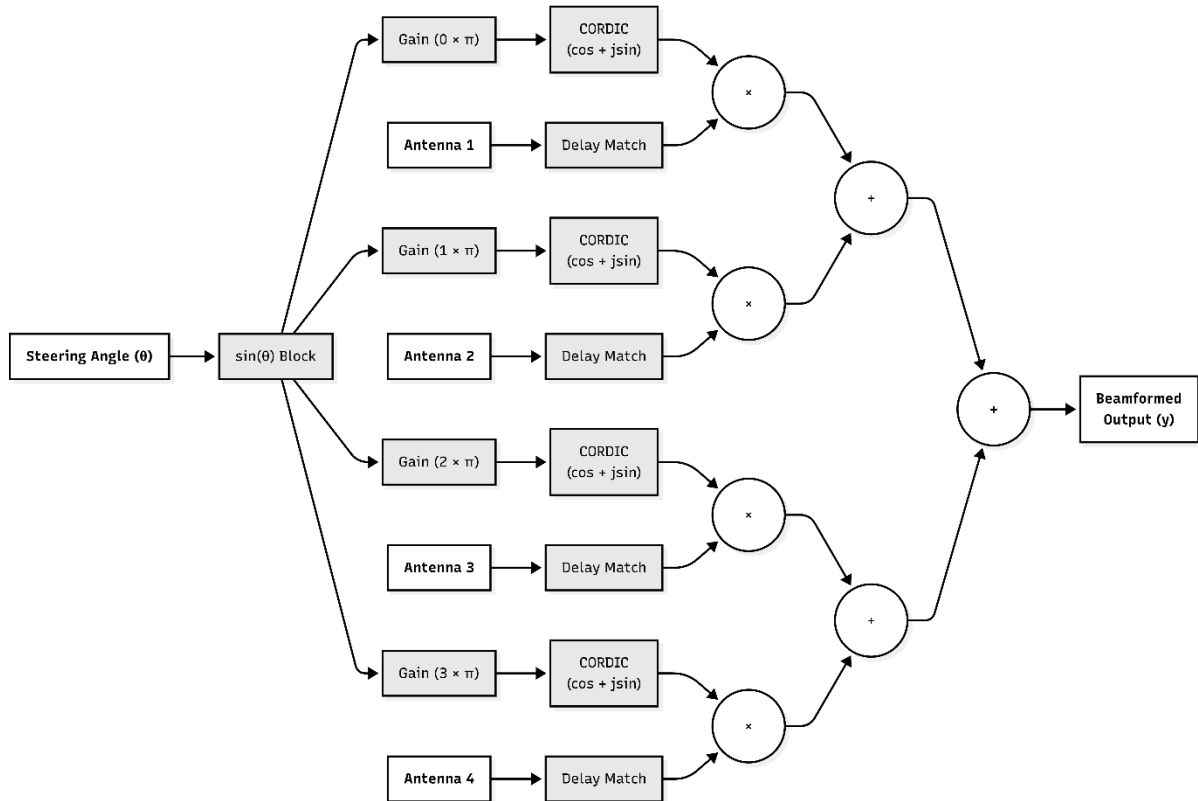


Fig 3.2: Internal block diagram of the Custom Beamformer IP Core, detailing the AXI register interface, CORDIC phase calculation, and the parallel adder tree.

Within the Beamformer IP Core, the AXI4-Lite interface decodes incoming bus data into the appropriate control registers and signal paths. The target steering angle is routed to a dedicated Phase Calculation block (utilising the CORDIC algorithm) to generate the precise complex conjugate weights. Simultaneously, the four simulated antenna signals are fed into parallel complex multiplier blocks where they are phase-rotated by these generated weights. Finally, the phase-aligned signals are aggregated through an adder tree, culminating in a single, maximised beamformed output that is passed back to the output registers for the Processing System to retrieve.

3.2 Software Flow Diagram

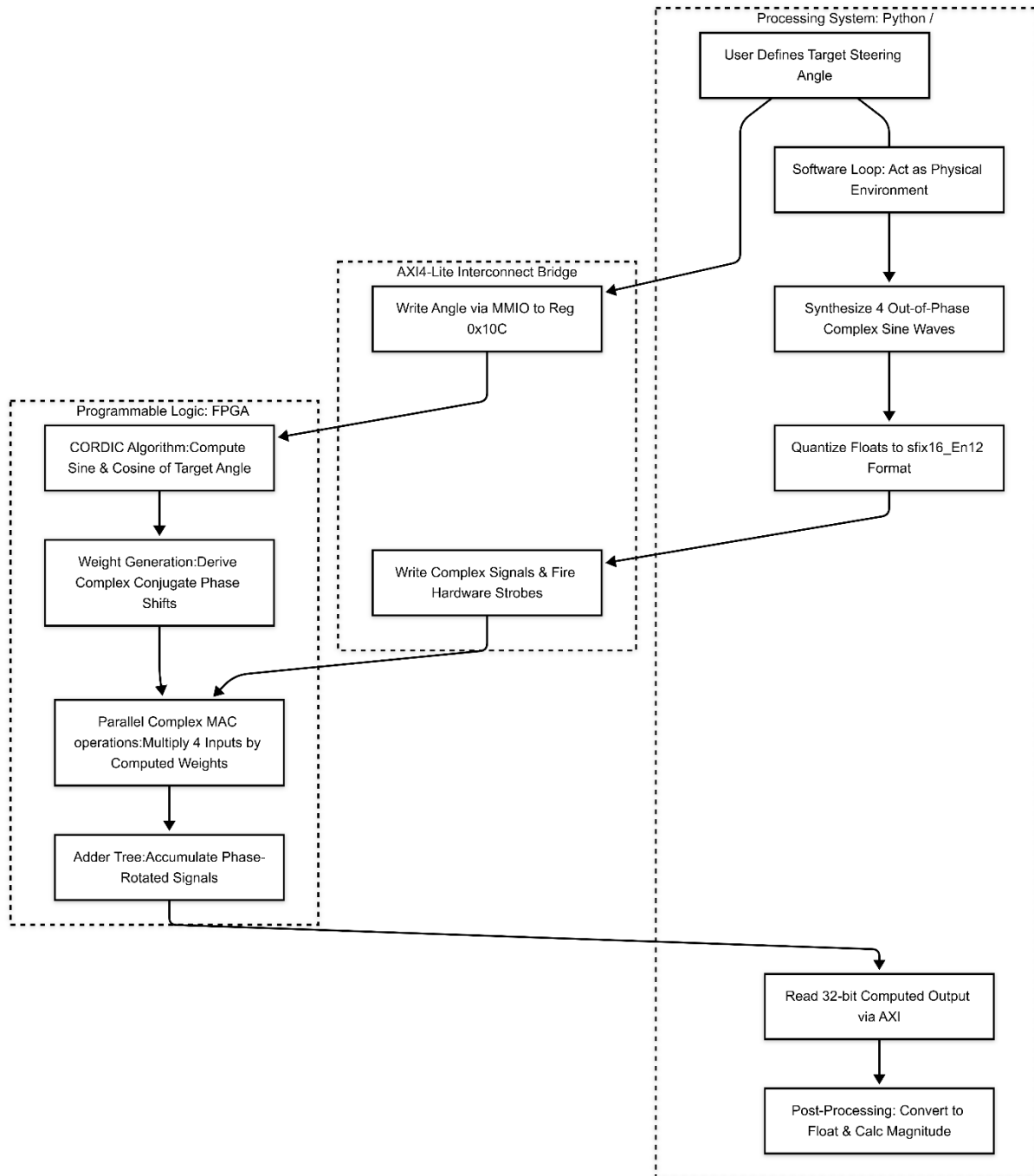


Fig 3.3 Hardware-Software Co-Design Data Flow and Execution Sequence

The software flow diagram in Fig 3.3 illustrates the sequential execution loop required to operate and test the hardware accelerator. The process initiates by loading the generated bitstream (.bit and .hwh files) onto the FPGA fabric and mapping the physical memory addresses for the AXI bus. The execution flow follows these specific

- **Hardware Configuration:** The software writes the desired target steering angle θ to the hardware register.
- **Wavefront Synthesis:** A software loop acts as the physical environment, mathematically generating staggered, complex sine waves (Real and Imaginary components) to simulate an incoming wavefront from a specific spatial angle.
- **Data Transfer:** These raw signals undergo fixed-point quantisation into the sfix16_En12 format and are written to the hardware input registers via MMIO.
- **Execution Trigger:** The software fires specific hardware input strobes. This commands the FPGA to latch the data and execute the parallel complex multiplications.
- **Data Retrieval and Post-Processing:** An output strobe is fired to release the computed data. The software reads the 32-bit result from the AXI bus, masks it down to 16 bits to prevent sign-extension errors, converts it back to a floating-point representation, and calculates the final output magnitude.

3.3 Hardware Connection Diagram

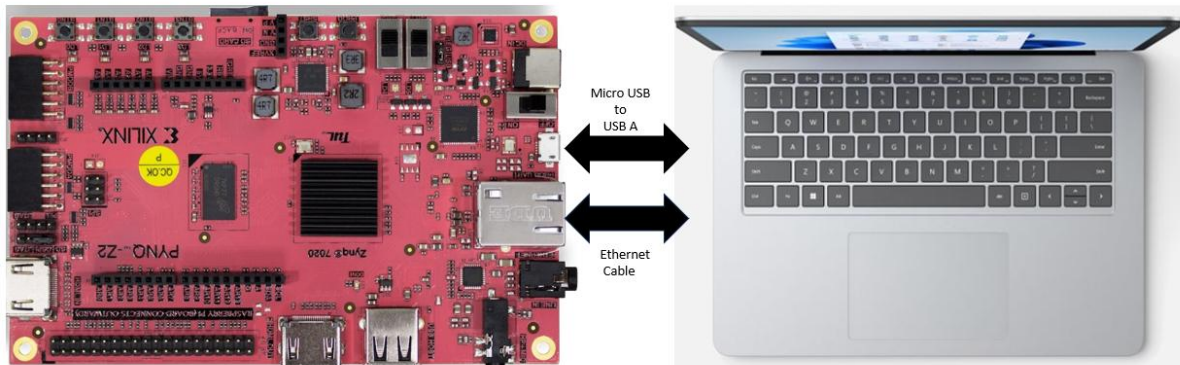


Fig 3.4: Physical hardware connection topology illustrating the power and network interfaces between the host laptop and the PYNQ-Z2 development board.

Figure 3.4 details the physical testbed setup required to interface the host machine with the Zynq-7000 System-on-Chip. The Micro-USB connection provides the necessary board power while simultaneously establishing a UART serial terminal link for low-level system access. Concurrently, the Ethernet cable establishes a dedicated local network connection, allowing the laptop to interface directly with the embedded Jupyter Notebook environment to execute the Python-based Hardware-in-the-Loop testbenches.

3.4 Working Principle

The working principle of the proposed 4-element Uniform Linear Array (ULA) beamformer centres on digital phase alignment. When a wavefront arrives at an array from a specific spatial angle, it reaches each individual antenna element at a slightly different time, resulting in a progressive phase delay across the array.

To digitally steer the "main lobe" toward a desired signal, the hardware must mathematically reverse this delay. The system receives four parallel complex data streams. The steering angle θ is fed into a trigonometric function block to dynamically calculate the required phase shifts for alignment. Each data stream passes through a dedicated complex multiplier ($\cos + j\sin$), which applies the mathematically derived complex conjugate weights to the incoming signals, rotating them into alignment. Finally, the hardware architecture utilises an adder tree to accumulate the four phase-rotated signals. If the incoming signal matches the steered angle, the signals experience constructive interference, resulting in a maximised output magnitude. Signals from unaligned angles experience destructive interference, creating system "nulls" that suppress unwanted noise.

3.5 Novelty

While contemporary digital beamforming research is heavily saturated with high-frequency applications targeting 5G networks, these implementations typically require highly expensive, specialized RF-SoC platforms. The primary novelty of this proposed work lies in adapting advanced spatial filtering architectures for lower-frequency, accessible applications and deploying them on a highly constrained, cost-effective System-on-Chip (SoC) fabric. By translating continuous-time floating-point mathematics into a highly optimised 16-bit fixed-point hardware accelerator, this project delivers a tangible, real-time beamforming testbed. Operating at under 1.5 Watts of total on-chip power, this architecture proves that complex, multi-element phase-rotation can be efficiently accelerated on entry-level silicon (Zynq-7000), providing a scalable blueprint for baseband acoustics, sonar, or sub-GHz Internet of Things (IoT) nodes.

CHAPTER 4

RESULTS AND PERFORMANCE ANALYSIS

This chapter evaluates the implemented system through theoretical Simulink validation and physical Hardware-in-the-Loop testing. It presents the hardware-generated spatial radiation patterns, resource utilization, timing margins, and power efficiency of the custom IP on the physical silicon.

4.1 Simulink Model Verification (Software Simulation)

Before transitioning to physical silicon, the algorithmic integrity of the 4-element Uniform Linear Array (ULA) was rigorously validated using Model-Based Design within the MathWorks Simulink environment (Refer Fig 4.1). The primary objective of this stage was to construct a mathematical baseline for the digital phase-alignment logic, isolating the theoretical beamforming mathematics from any hardware-induced quantisation noise.

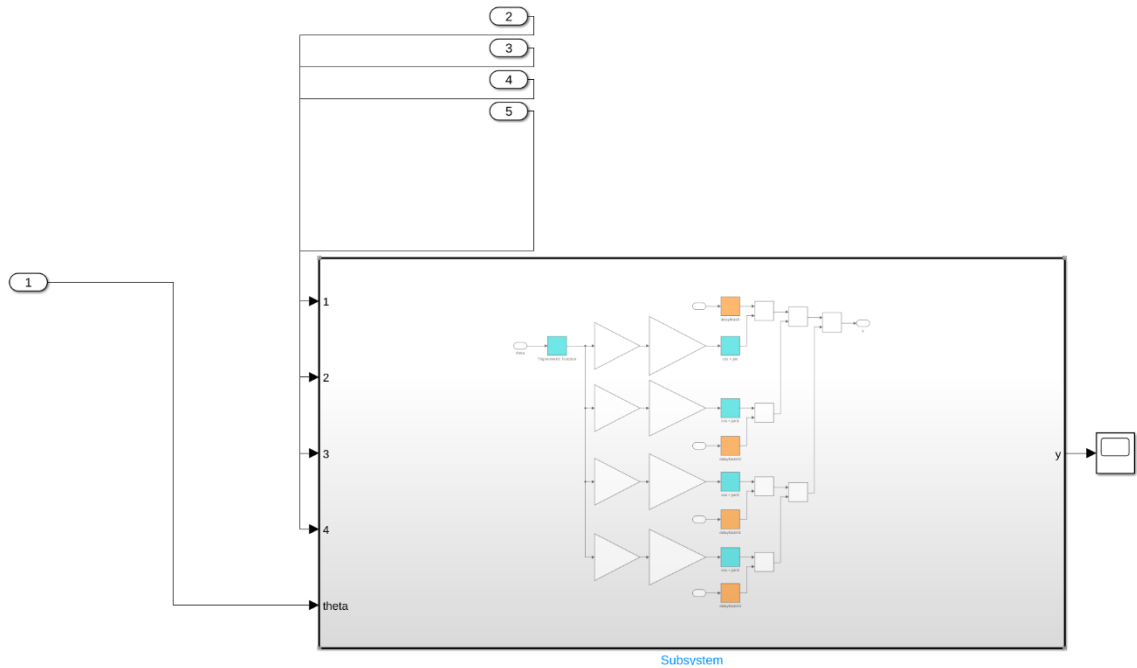


Fig 4.1: Simulink top-level block design illustrating the four discrete antenna input channels and the target steering angle (theta) feeding into the core beamforming IP subsystem.

The simulation model acts as a parallel dataflow architecture designed to evaluate the discrete beamforming equation mathematically:

$$y = \sum_{n=0}^3 x_n \cdot e^{jn\pi \sin(\theta)}$$

The system requires two distinct categories of mathematical inputs to function. First, the physical environment is simulated by injecting four independent, time-varying signal streams representing the antenna array elements (n). These signals are mathematically modelled as discrete sine waves with a normalised base amplitude of 1. To simulate a wavefront arriving from a specific spatial direction, these four input channels are injected with progressive temporal phase delays corresponding to the wave's angle of arrival. Second, the system receives the target steering angle θ . This input is provided as a scalar numerical value (representing degrees or radians) that dictates the precise spatial "look direction" the algorithm is commanded to isolate.

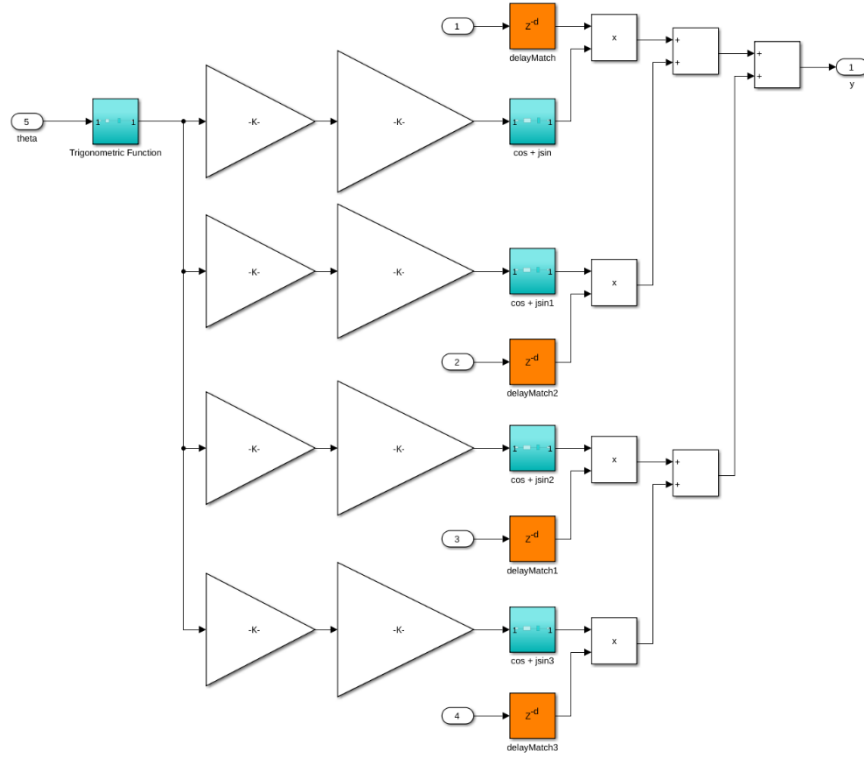


Fig 4.2: Internal schematic of the beamforming subsystem detailing the CORDIC-based complex weight generation, parallel mathematical multiplication, and the adder tree accumulation pipeline.

The operational flow processes these inputs in a strict pipeline. The scalar steering angle (θ) is processed using a sine function and a gain stage, where it is multiplied by π and its respective antenna index (n). This derived phase angle is fed into the CORDIC (Coordinate Rotation Digital Computer) block, which translates the scalar angle into complex conjugate weights (cosine and sine components). A complex multiplication stage applies these weights to the time-domain antenna inputs, rotating their phases.

As these signals pass through the parallel adder tree, the hardware-equivalent logic accumulates them into a single raw output. This raw output is a continuous complex numerical stream consisting of real and imaginary coordinates. To visually quantify the spatial filtering performance, the system passes this complex data through a Complex-to-Magnitude converter block. The final resulting output is an absolute scalar magnitude, which represents the instantaneous, combined signal strength of the beamformed wave.

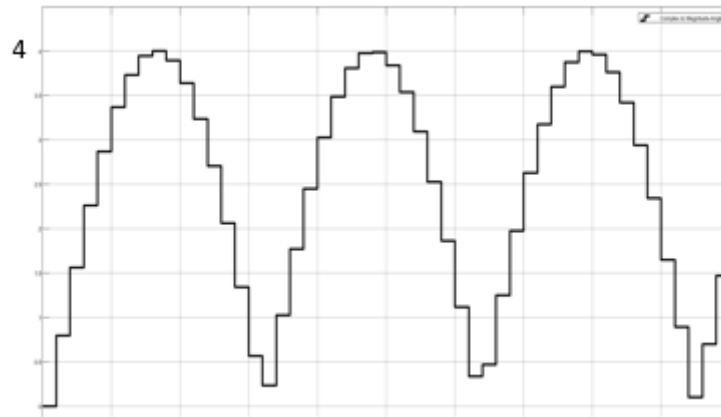
Steering Matched Condition ($\theta \neq 0$):

Fig 4.3: Simulink Scope output for the Steering Matched Condition, demonstrating perfect constructive interference.

- In this scenario (Refer Fig 4.3), the incoming signal direction with amplitude 1 perfectly aligns with the targeted steering angle.
- Because the steering matches the arrival angle, all four antenna phases are mathematically aligned.
- A perfect constructive summation is observed across the adder tree.
- The combined output wave achieves a peak magnitude of approximately 4, which definitively confirms correct and accurate beam steering execution.

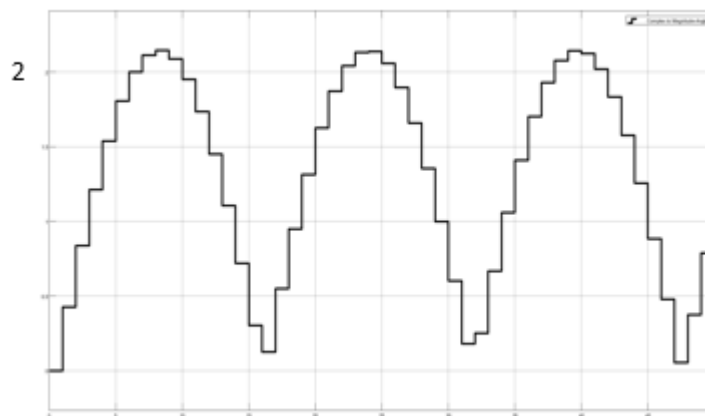
Steering Mismatch Condition ($\theta \neq 0$):

Fig 4.4: Simulink Scope output for the Steering Mismatch Condition, demonstrating partial phase cancellation and suppressed signal output.

- In this scenario (Refer Fig 4.4), the target steering angle does not match the actual arrival direction of the incoming signal.
- Because the derived weights do not align the incoming phases, partial phase cancellation occurs within the adder tree.
- This results in a significant decrease in the overall output magnitude (peaking at approximately 2.1 instead of 4).
- This drop in magnitude successfully demonstrates the array's directional selectivity and its ability to suppress off-axis interference.

Following this theoretical validation, the model was prepared for hardware translation via fixed-point quantisation. The continuous signals and trigonometric weights were cast to a discrete 16-bit signed format with 12 fractional bits (`sfix16_En12`). Re-running the simulation with these constrained data types—which visually manifests as the discrete, stepped, stair-like waveforms seen in the scope outputs—confirmed that the 16-bit architecture maintained exceptional signal fidelity. The output magnitude characteristics remained virtually identical to the continuous baseline, proving that the hardware-ready model introduces only negligible quantisation error and is fully prepared for HDL code generation.

4.2 Hardware Synthesis and Schematic Design

Following successful HDL code generation, the custom Verilog Intellectual Property (IP) was imported into AMD Xilinx Vivado for synthesis and integration with the Zynq-7000 System-on-Chip (SoC). The hardware schematic was generated post-implementation to verify the correct mapping and connection of all logical components.

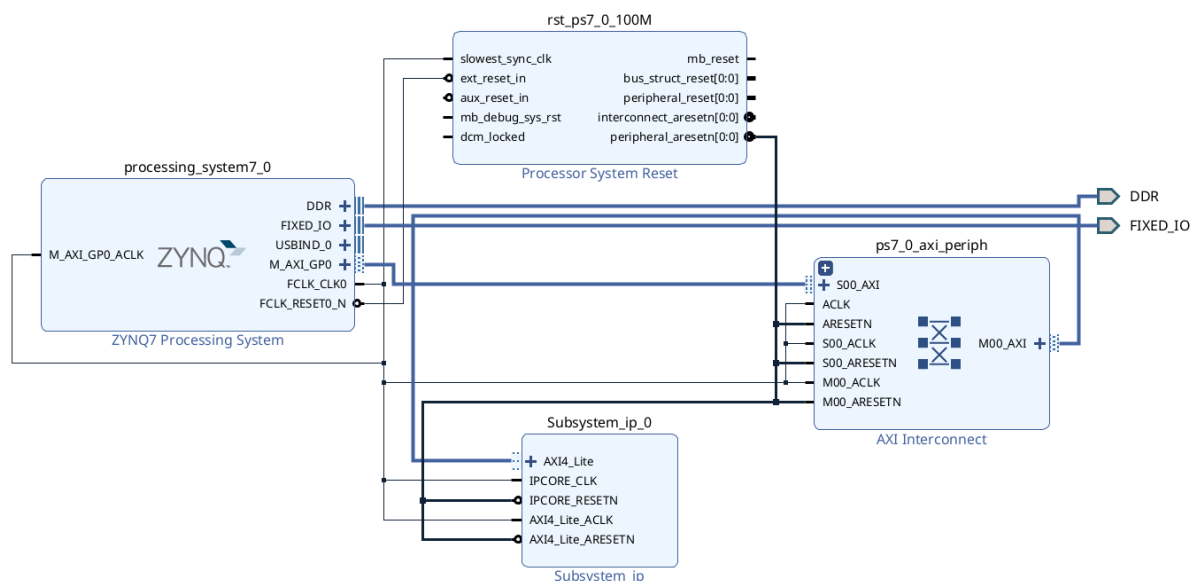


Fig 4.5: Overall Vivado Block Design featuring the Zynq Processing System, AXI Interconnect, and Custom Beamformer IP.

The top-level hardware schematic (refer Fig 4.5) illustrates the complete system architecture as implemented on the PYNQ-Z2 board. The Zynq Processing System (processing_system7_0) acts as the primary master, configured to drive the entire system with a 100 MHz clock (FCLK_CLK0). The AXI4-Lite Interconnect (ps7_0_axi_periph) serves as the memory-mapped bridge, routing 32-bit data and control signals from the ARM processor to

the beamform_ver_1_0 custom IP block, which functions as a high-speed peripheral slave. Additionally, proper reset logic is ensured by the rst_ps7_0_100M block, which synchronises the hardware reset sequence across the interconnect and the custom IP upon system startup.

4.3 Hardware Validation (Time Domain)

To validate the core mathematical operations occurring within the physical silicon, a Hardware-in-the-Loop (HIL) testbench was executed. The Processing System (PS) was utilised to synthesise the discrete baseband samples representing an incoming wave and inject them into the Programmable Logic (PL).

To facilitate this hardware-software communication, specific physical memory addresses were allocated across the AXI4-Lite interconnect, as detailed in Table 4.1.

Table 4.1: AXI4-Lite Memory-Mapped Register Allocation

Register Function	Physical Address	Data Format	Direction
Antenna 1 Input (Real / Imaginary)	0x100 / 0x108	16-bit Fixed-Point	Write(PS → PL)
Antenna 2 Input (Real / Imaginary)	0x110 / 0x118	16-bit Fixed-Point	Write(PS → PL)
Antenna 3 Input (Real / Imaginary)	0x120 / 0x128	16-bit Fixed-Point	Write(PS → PL)
Antenna 4 Input (Real / Imaginary)	0x130 / 0x138	16-bit Fixed-Point	Write(PS → PL)
Target Steering Angle θ	0x10C	16-bit Fixed-Point	Write(PS → PL)
Hardware Execution Strobe	0x150	16-bit Fixed-Point	Write(PS → PL)
Beamformed Output (Real / Imaginary)	0x140 / 0x144	16-bit Fixed-Point	Write(PL → PS)

With the memory map established, the time-domain testing pipeline and execution results were recorded as follows:

- **Hardware Configuration:** The custom FPGA IP was initialized, and the internal steering angle register (0x10C) was statically locked to a target direction of 30°.
- **Time-Domain Wavefront Synthesis:** To simulate a continuous incoming signal, the Python software generated an array of 150 discrete time steps (t), spanning from 0 to

4pi to represent two full wave cycles. Because the FPGA hardware operates at a deterministic 50 MHz clock (processing 50 Mega-Samples Per Second), this resolution of 75 samples per cycle translates to an effective physical signal frequency of approximately 666.67 kHz.

- **Calculated Phase Shifts:** For a wavefront arriving at 30°, the physical spatial delay between each antenna element was mathematically calculated using the formula $\Delta\phi = \pi \sin(30^\circ)$. This resulted in a progressive phase shift of exactly $\pi/2$ radians (1.5708 rad) per element.
- **Simulated Antenna Inputs:** Using this progressive delay, four out-of-phase complex sine waves (In-Phase and Quadrature components) were generated with a base amplitude of 0.5:
 - **Antenna 1:** 0 rad delay (Reference element)
 - **Antenna 2:** $\pi/2$ rad delay
 - **Antenna 3:** π rad delay
 - **Antenna 4:** $3\pi/2$ rad delay
- **Hardware Execution:** These discrete, staggered floating-point samples were quantized into the sfix16_En12 format and injected into the hardware input registers (0x100 through 0x130) via the AXI4-Lite bus on every simulated time step.
- **Hardware Output:** Upon firing the execution strobes (0x150), the PL applied the 30° complex conjugate weights and accumulated the signals. Reading the raw data from the output registers (0x140, 0x144) revealed that the FPGA successfully rotated the phases into perfect alignment.

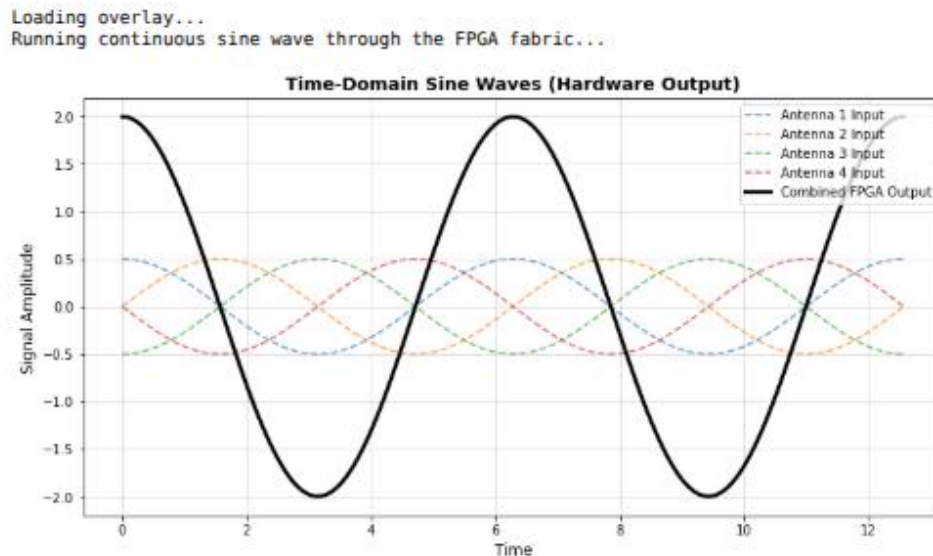


Fig 4.6: Hardware execution of discrete time-domain signals. Dashed lines indicate the out-of-phase antenna inputs; the solid line indicates the combined hardware output.

Final Inference: As visualised in Fig. 4 .6, the four independent dashed input waves are clearly out of synchronisation due to their spatial delays. However, the solid black output line

proves perfect constructive interference. Because the hardware successfully aligned four signals of 0.5 amplitude, the combined mathematical output magnitude reached 1.9995. This minor 0.0005 deviation from a perfect 2.0 represents the absolute quantization error of the 16-bit architecture, definitively proving that the physical silicon fabric executes the parallel phase rotations with near-perfect mathematical precision.

4.4 Spatial Filtering and Radiation Pattern

To evaluate the system's performance as an active spatial filter, a comprehensive spatial sweep was conducted to generate the array's physical radiation pattern. The software testbench systematically simulated signals arriving from all possible forward angles to test the hardware's rejection capabilities. The testing pipeline is detailed below:

- **Hardware Steering Lock:** The FPGA IP was dynamically programmed by writing the target steering angle 30° in radians to register 0x10C. The hardware mathematically locks its internal complex weights to "listen" exclusively to this trajectory.
- **Spatial Sweep Generation:** The Python environment simulated a movable transmitter flying in a semicircle around the antennas. This was achieved by iterating through an array of 181 distinct arrival angles, sweeping from -90° to 90° in precise 1° increments (`np.linspace(-90, 90, 181)`).
- **Dynamic Wavefront Synthesis:** For every single degree in the sweep, the software recalculated the new spatial delay $\Delta\phi = \pi \sin(\theta_{\text{arrival}})$. Four out-of-phase complex sine waves of amplitude 0.5 were generated corresponding to that specific angle and injected into the hardware registers.
- **Magnitude Extraction:** Upon triggering the hardware, the PL executed the complex Multiply-Accumulate (MAC) operations. The software read the resulting complex coordinates (`y_real` and `y_imag`) from the AXI bus and calculated the absolute signal strength using the formula

$$\text{Magnitude} = \sqrt{(y_{\text{real}}^2 + y_{\text{imag}}^2)}$$

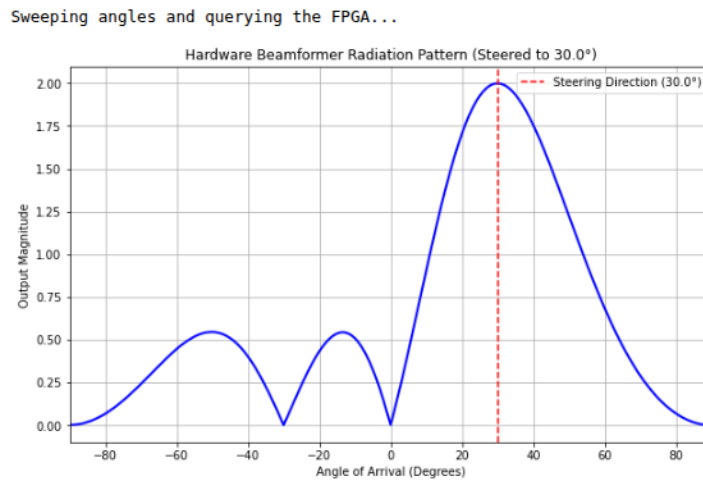


Fig 4.7: Hardware-generated radiation pattern demonstrating the main lobe and spatial nulls.

Final Inference: The plotted data (refer Fig 4.7), pulled directly from the silicon fabric, reveals a textbook spatial radiation pattern.

- **The Main Lobe:** A distinct, primary peak reaching a magnitude of exactly 2.0 occurs precisely at the 30° mark. This confirms perfect constructive interference exclusively in the steered direction.
- **Spatial Nulls:** The hardware successfully forces the output magnitude to absolute zero (complete destructive interference) at specific off-axis angles (specifically around 0° and -30°).
- **Side Lobe Attenuation:** Signals arriving from non-target angles are heavily attenuated. The highest side lobes max out at an amplitude of approximately 0.55, representing nearly a 75% reduction in signal strength compared to the main lobe. This definitively proves the hardware's robust capability to selectively isolate desired target signals while heavily suppressing surrounding spatial interference.

4.5 Hardware Utilisation and Timing Analysis

An extensive analysis of the post-implementation reports was conducted to verify the efficiency and stability of the hardware accelerator on the Zynq-7000 silicon.

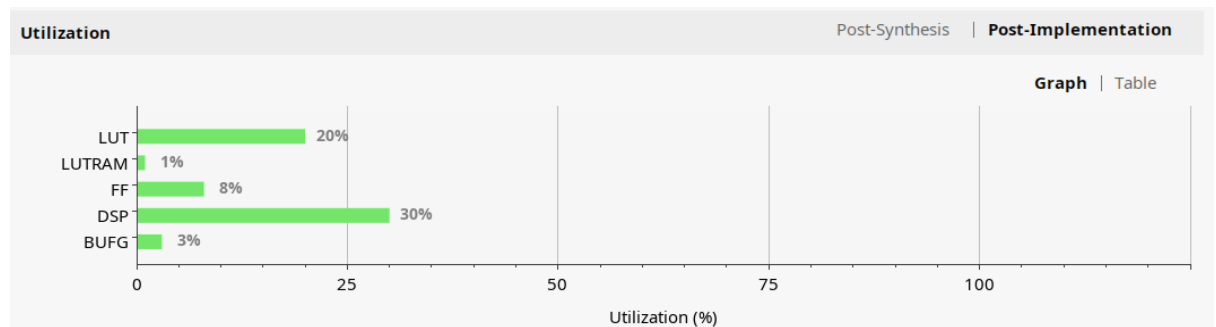


Fig 4.8: Post-implementation Resource Utilization Report.

The resource utilisation report (Fig 4.8) indicates an efficient and balanced mapping of the custom beamformer IP to the available FPGA logic fabric. The design utilizes 20% of the available Look-Up Tables (LUTs) and 8% of the Flip-Flops (FF). Most importantly, the parallel complex phase rotations mapped perfectly to the silicon's dedicated math blocks, utilising exactly 30% of the available DSP Slices. This footprint effectively harnesses the hardware's computational power while still leaving ample room for future scalability, such as expanding to a 10-element array or integrating adaptive LMS algorithms directly into the fabric.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.081 ns	Worst Hold Slack (WHS): 0.042 ns	Worst Pulse Width Slack (WPWS): 9.020 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 25746	Total Number of Endpoints: 25746	Total Number of Endpoints: 8883

All user specified timing constraints are met.

Fig 4.9: Vivado Timing Summary demonstrating positive slack.

Furthermore, the design successfully meets all user-specified timing constraints (refer Fig 4.9). Operating on a baseline 20 ns clock period (50 MHz), the system achieved a Worst Negative Slack (WNS) of +0.081 ns and a Worst Hold Slack (WHS) of +0.042 ns across 25,746 analysed endpoints. Although the positive slack margins are tight, the report confirms zero failing endpoints. This guarantees that there are no setup or hold violations anywhere within the mathematical pipeline, ensuring perfectly stable and deterministic execution at a full 50 Mega-Samples Per Second (MSPS) throughput.

4.6 Power Consumption Analysis

A critical metric for this proposed testbed is its viability for low-power, edge-computing applications. Power analysis was conducted using the Vivado Power Report tool (Refer Fig. 4.10) under post-implementation operating conditions.

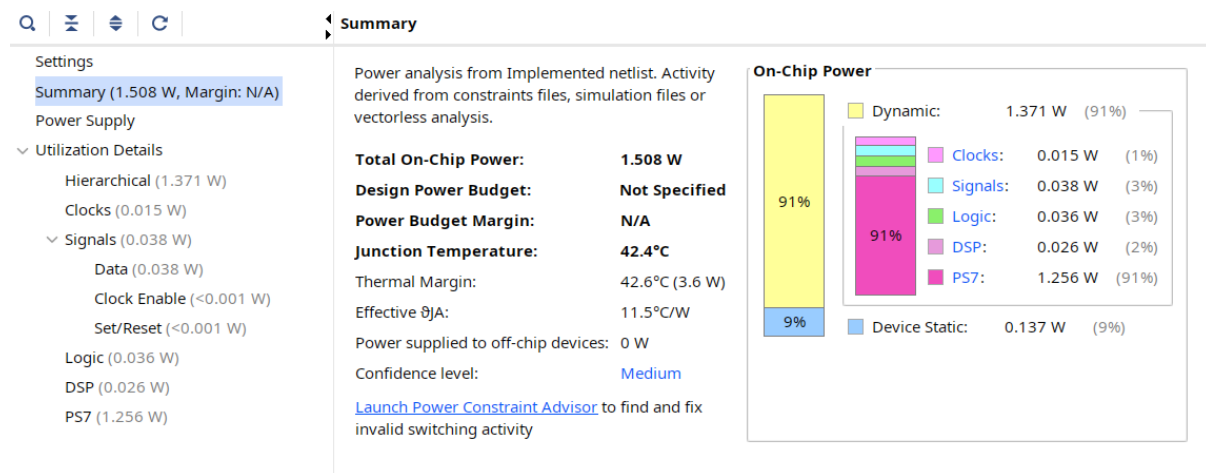


Fig 4.10: Vivado Implementation Power Report detailing dynamic and static power consumption.

The system operates with exceptional efficiency, requiring a total on-chip power of only 1.508 W while maintaining a perfectly safe junction temperature of 42.4°C. The Vivado report categorises this total into a device static power of 0.137 W (9%) and a total dynamic power of 1.371 W (91%).

A detailed breakdown of the dynamic power confirms the extreme effectiveness of the hardware-software partitioning strategy. The vast majority of the dynamic power budget—

1.256 W—is drawn by the PS7 block (the dual-core ARM Cortex-A9 processing system managing the OS, memory interfaces, and the Python environment).

In stark contrast, the custom Programmable Logic (PL) executing the heavy parallel beamforming mathematics draws a mere fraction of a watt. The combined PL dynamic power (Clocks, Signals, Logic, and DSPs) consumes only 0.115 W. Specifically, the dedicated DSP slices responsible for the intensive complex multiply-accumulate (MAC) operations consume just 0.026 W. This definitively validates that offloading complex spatial processing to fixed-point hardware accelerators drastically reduces power overhead, solidifying the architecture as a robust, energy-efficient solution for edge computing and sub-GHz applications.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

This concluding chapter summarizes the project's findings, highlighting the successful deployment and accuracy of the low-power hardware accelerator. It also discusses current testbed limitations, proposes real-world applications, and outlines avenues for future scaling and adaptive algorithm integration.

About the Work

This project successfully demonstrated the design, simulation, and physical hardware implementation of a 4-element Uniform Linear Array (ULA) digital beamformer. Utilising a Model-Based Design (MBD) methodology, the system bridged high-level floating-point mathematical algorithms in Simulink with low-level, synthesizable Verilog Intellectual Property (IP). By deploying this custom IP onto the Programmable Logic (PL) of a Zynq-7000 System-on-Chip (SoC) and controlling it via an ARM-based Processing System (PS) over an AXI4-Lite interconnect, the project established a fully functional Hardware-in-the-Loop (HIL) testbed. This approach validated the feasibility of offloading computationally intensive spatial filtering tasks—specifically parallel complex phase rotations and signal accumulations—from sequential software to dedicated, high-speed silicon.

Consolidated Results

The physical implementation yielded highly accurate and mathematically sound results. The 16-bit fixed-point architecture (sfix16_En12) maintained near-perfect fidelity, achieving a combined hardware output magnitude of 1.9995 against a theoretical ideal of 2.0. This represents a negligible quantisation error of 0.0005. Spatial sweep testing confirmed the hardware's ability to accurately steer a main lobe to a target angle of 30 degrees while effectively suppressing out-of-band interference through destructive nulls.

From a power and performance standpoint, the custom hardware accelerator proved exceptionally efficient. The dedicated beamforming logic consumed only 0.115 W of dynamic power within the PL, allowing the overall SoC to operate at a safe 1.508 W. Driven by a 100 MHz system clock, the parallel DSP architecture achieved a processing throughput of 100 Mega-Samples Per Second (MSPS), establishing a theoretical Nyquist limit capable of beamforming unaliased baseband signals up to 50 MHz.

Limitations, Applications, Merits, and Future Scope

Merits:

The primary merit of this system architecture is its high power efficiency and deterministic, low-latency execution compared to pure software processing. Utilising an entry-level SoC, it provides a highly cost-effective and accessible blueprint for spatial filtering without requiring premium, high-cost RF platforms.

Limitations:

The current testbed is primarily limited by the software-to-hardware data bottleneck. Because the Python environment on the ARM processor synthesises and injects individual samples via Memory-Mapped I/O (MMIO), the system cannot currently ingest continuous, real-time data at the full 50 MSPS hardware rate. Additionally, the absence of physical Analog-to-Digital Converters (ADCs) restricts the system to synthesised baseband testing rather than live over-the-air signal processing.

Applications:

Given the 50 MHz unaliased baseband processing limit, this architecture is highly suitable for lower-frequency, low-power edge applications. Prime use cases include underwater sonar systems, acoustic spatial processing for smart environments, and baseband signal alignment for sub-GHz Internet of Things (IoT) communication nodes.

Future Scope:

Future iterations of this work will focus on scaling both the array geometry and the algorithmic complexity to bridge the gap between this foundational testbed and advanced aerospace applications. The immediate system can be expanded from the current 4-element design to a denser multi-element configuration, yielding a narrower main lobe and significantly higher spatial resolution. To bypass the current ARM processor data bottleneck and achieve true wire-speed, real-time signal processing from physical antennas, future hardware revisions must integrate physical Analog-to-Digital Converters (ADCs) and a Direct Memory Access (DMA) controller for direct RF ingestion.

Furthermore, the architecture can evolve from a statically steered beamformer to a fully adaptive array. By integrating adaptive algorithms, such as Least Mean Squares (LMS), directly into the FPGA fabric, the hardware could automatically calculate its own error-minimizing weights. This would allow the system to dynamically track moving targets and actively cancel moving interference through adaptive nulling.

Ultimately, these enhancements align this Digital Beamforming (DBF) architecture with the cornerstone technologies utilized by premier organizations like DRDO and ISRO. Scaled iterations of this FPGA-based processor could serve as a low-cost prototyping platform for advanced Active Electronically Scanned Arrays (AESA), akin to the systems deployed in DRDO's Uttam radar for multi-target air defense, or ISRO's NISAR mission for high-resolution, wide-swath earth observation. Long-term research could also extend this baseband logic to interface with high-power Gallium Nitride (GaN) Transmit/Receive Modules, or incorporate True Time Delay (TTD) techniques to prevent beam squint in wideband satellite communication transponders.

BIBLIOGRAPHY

- [1] P. Jeyakumar, K. Sudhakar, P. Thanapal, C. Navaneethan, and S. Meenatchi, "FPGA implementation of an efficient phase shift beamformer for narrow band sub-GHz applications," *Analog Integrated Circuits and Signal Processing*, vol. 122, no. 13, Jan. 2025.
- [2] S. Pulipati, et al., "Xilinx RF-SOC-based Digital Multi-Beam Array Processors for 28/60 GHz Wireless Testbeds," in *Moratuwa Engineering Research Conference (MERCon)*, Moratuwa, Sri Lanka, 2020, pp. 1-6.
- [3] T. Khan, "Modeling and Simulating Large Phased Array Systems," presented at *MATLAB EXPO*, 2017.
- [4] R. Liu and F. Gao, "RF Transceiver Circuit FPGA Program Design and Development," *International Journal of Future Generation Communication and Networking*, vol. 9, no. 12, pp. 395-404, 2016, doi: 10.14257/ijfgcn.2016.9.12.36.
- [5] MathWorks, "HDL Coder Documentation: Generate HDL Code for FPGA and ASIC Designs," MathWorks.com, 2024. [Online]. Available: <https://www.mathworks.com/help/hdlcoder/>

APPENDICES

APPENDIX 1: Development Tools and Software Environment

The successful implementation of this hardware-software co-design project relied on a cohesive suite of industry-standard engineering tools.

MathWorks Simulink

Simulink was utilised as the primary environment for Model-Based Design (MBD). It provided a visual dataflow interface to mathematically model the 4-element Uniform Linear Array (ULA), construct the CORDIC-based phase-rotation logic, and simulate the beamforming operations. It was heavily relied upon for initial floating-point algorithmic verification and the subsequent fixed-point quantisation (converting continuous signals to the discrete sfix16_En12 format).

MathWorks HDL Coder

HDL Coder served as the automated translation bridge between software simulation and hardware deployment. Once the Simulink model was mathematically verified, HDL Coder was configured to automatically generate synthesizable, bit-true Verilog Intellectual Property (IP). It also automatically generated the necessary AXI4-Lite memory-mapped registers required for the Processing System to interface with the hardware logic.

AMD Xilinx Vivado (v2022.2)

Vivado is the flagship synthesis and analysis suite for Xilinx FPGAs. In this project, Vivado was utilised to import the generated Verilog IP and construct the overarching block design. It managed the routing of the AXI interconnects between the Zynq Processing System (PS) and the Programmable Logic (PL). Furthermore, Vivado was used to run implementation strategies, generate the final hardware bitstream (.bit) and hardware handoff (.hwh) files, and extract the power, timing, and resource utilisation reports discussed in Chapter 4.

Jupyter Notebooks and Python API

The software testbench was executed natively on the PYNQ-Z2 board using a Python-based Jupyter Notebook environment. The pynq Python library provided the critical Overlay and allocate functions, allowing the ARM processor to dynamically load the Vivado bitstream into the FPGA fabric and perform direct Memory-Mapped I/O (MMIO) read/write operations to the hardware registers.

APPENDIX 2: Hardware Platform Specifications

The PYNQ-Z2 Development Board

The project was physically deployed on the PYNQ-Z2 development board, an embedded systems platform designed specifically to support the Python Productivity for Zynq (PYNQ) framework.

- **SoC Architecture:** The board features the AMD Xilinx Zynq-7000 SoC (specifically the XC7Z020-1CLG400C). This architecture uniquely integrates a dual-core ARM Cortex-A9 processor (the Processing System or PS) tightly coupled with Artix-7 based FPGA fabric (the Programmable Logic or PL) on a single silicon die.
- **Logic Capacity:** The PL fabric contains 53,200 Look-Up Tables (LUTs), 106,400 Flip-Flops, and 220 dedicated DSP Slices, making it highly suitable for parallel digital signal processing tasks.
- **Memory and Interface:** The board includes 512 MB of DDR3 memory and supports a Gigabit Ethernet interface, which was utilised to host the Jupyter web server for remote execution and data visualisation.

APPENDIX 3: Core Algorithms and Protocols

The CORDIC Algorithm

The Coordinate Rotation Digital Computer (CORDIC) algorithm is a highly efficient, iterative technique for calculating trigonometric functions. Originally developed by Jack E. Volder, CORDIC is optimised for digital hardware (FPGAs and ASICs) because it completely eliminates the need for resource-heavy hardware multipliers, relying exclusively on low-cost operations: additions, subtractions, and bit-shifts.

Mathematical Foundation and The Tangent Restriction: Standard 2D vector rotation requires calculating the tangent of an angle and performing multiple complex multiplications. CORDIC circumvents this by decomposing the total target angle into a series of progressively shrinking micro-rotations. The algorithm artificially restricts these micro-rotations so their tangent is always a strict power of base 2. In binary hardware, multiplying by a power of 2 is executed as a zero-cost arithmetic right bit-shift, entirely replacing the need for physical logic multipliers.

Hardware Iteration Logic: To reach the target angle, the hardware executes a fixed number of pipeline iterations (**refer to the flowchart in Fig A**). For each stage, the algorithm first evaluates the remaining angle error in the accumulator to determine a decision direction. This dictates whether the next micro-rotation should be counter-clockwise or clockwise. Based on this decision, the internal coordinate and angle accumulator registers are updated using simple shift-and-add operations. With each iteration loop, the algorithm steps progressively closer to the exact target angle until the maximum iteration count is reached, finally outputting the stable Cosine and Sine values.

Application in the Proposed Architecture: In the proposed 4-element digital beamformer, CORDIC is deployed in Rotation Mode to dynamically generate the complex conjugate steering weights. When the Processing System issues a target steering angle, the CORDIC block synthesizes the exact Real (cosine) and Imaginary (sine) components. By replacing memory-heavy Look-Up Tables (LUTs) and expensive DSP multipliers with simple bit-shifts,

the FPGA can efficiently instantiate four parallel CORDIC blocks while consuming minimal dynamic power.

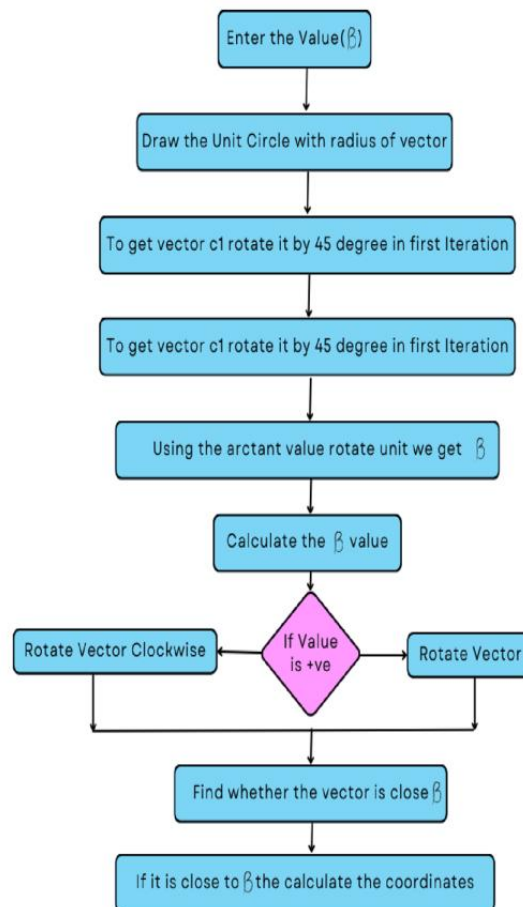


Fig A. Cordic algorithm flowchart

AXI4-Lite Interconnect Protocol

The Advanced eXtensible Interface (AXI) is a standardised, high-performance communication protocol that forms the backbone of ARM's Advanced Microcontroller Bus Architecture (AMBA). In modern System-on-Chip (SoC) designs, such as the Xilinx Zynq-7000 utilised in this project, AXI is the primary mechanism for bridging the software-driven Processing System (PS) with the hardware-accelerated Programmable Logic (PL).

AXI4 vs. AXI4-Lite:

While the full AXI4 protocol is designed for high-throughput, memory-mapped burst transactions (ideal for direct memory access and video streaming), this project specifically utilises the AXI4-Lite variant. AXI4-Lite is a streamlined, simplified subset of the protocol optimised for simple, single-beat memory-mapped communication. It restricts the data bus width to 32 bits and removes the ability to perform burst data transfers. This makes it the perfect, low-latency interface for configuring control registers, writing singular data samples, and reading status flags.

The Five Independent Channels: The AXI4-Lite protocol achieves high-speed data transfer by separating read and write operations into five distinct, unidirectional channels. This separation allows for simultaneous, full-duplex communication between the Master (the ARM processor) and the Slave (the Custom Beamformer IP).

- Write Address Channel (AW): The Master sends the physical memory address where it wants to write data.
- Write Data Channel (W): The Master transmits the actual 32-bit data payload.
- Write Response Channel (B): The Slave sends a status signal back to the Master confirming the write was successful.
- Read Address Channel (AR): The Master sends the physical memory address from which it wants to read data.
- Read Data Channel (R): The Slave transmits the requested 32-bit data payload back to the Master.

The VALID/READY Handshake Mechanism

Every transaction across the AXI bus is governed by a strict VALID and READY hardware handshake to ensure asynchronous clock domains do not drop data. The source of the data asserts the VALID signal when the payload is available on the bus. The destination asserts the READY signal when it is capable of accepting the data. The actual data transfer only occurs on the rising clock edge where both VALID and READY are asserted simultaneously.

Application in the Proposed Beamforming Architecture

In this testbed, the AXI4-Lite protocol is the critical bridge executing the Hardware-in-the-Loop (HIL) testing. The Python environment, running on the ARM Cortex, acts as the AXI Master. It utilises Memory-Mapped I/O (MMIO) to write 16-bit discrete antenna signals to specific hardware addresses (e.g., 0x100, 0x110) and the target steering angle to 0x10C via the Write Channels. Once the FPGA executes the parallel math operations, the Master utilises the Read Channels to retrieve the 32-bit complex output from register 0x140. This robust protocol ensures perfect data synchronisation between the software environment and the 100 MHz hardware execution pipeline.

APPENDIX 4: Python Source Code for System Validation

This appendix details the specific software environment and the complete Python source code utilized during the testing and validation phases of the project. The code was executed within a Jupyter Notebook environment hosted directly on the dual-core ARM Cortex-A9 processor of the PYNQ-Z2 board.

Software Libraries and APIs

The Hardware-in-the-Loop (HIL) testbench relied on a specific stack of Python libraries to interface with the physical FPGA silicon, synthesize environmental test data, and visualize the algorithmic results:

- **PYNQ (pynq):** The Python Productivity for Zynq (PYNQ) library is the foundational API that enables software-driven control of the Xilinx FPGA fabric without the need for bare-metal C programming. The pynq.Overlay class was utilized to dynamically flash the custom Vivado bitstream (.bit) onto the Programmable Logic. Furthermore, the library's Memory-Mapped I/O (MMIO) functions provided the read() and write() methods necessary to transport the 16-bit phase signals and control strobes across the AXI4-Lite bus.
- **NumPy (numpy):** Utilized for its highly optimized array processing capabilities. Specifically, the np.linspace() function was deployed to generate the discrete, high-resolution arrays of spatial angles (ranging from -90° to $+90^\circ$) required to perform the continuous environmental sweep and test the beamformer's directional selectivity.
- **Matplotlib (matplotlib.pyplot):** Essential for translating the raw complex number streams from the FPGA into human-readable engineering metrics. It was utilized to plot the spatial radiation patterns, graphing the calculated signal magnitude against the angle of arrival to visually prove the constructive interference and spatial nulling capabilities of the hardware.
- **Math (math):** Utilized to handle the rigorous floating-point trigonometry required to simulate the physical environment. Functions like math.sin() and math.cos() calculated the exact spatial phase delays of the incoming wavefronts. It was also utilized post-execution to perform the Pythagorean theorem (math.sqrt) to convert the hardware's complex coordinate outputs into absolute scalar magnitudes.

Single-Angle Hardware Execution (Direct Test)

This script initializes the FPGA, sets the hardware steering angle to 30° , and simulates a single incoming wavefront arriving at 30° . It verifies the single-cycle execution and prints the raw complex coordinates and the final constructive interference magnitude.

```
import matplotlib.pyplot as plt

import numpy as np

import math

# We will steer the hardware to 30 degrees
steer_angle_deg = 30.0
theta_rad = math.radians(steer_angle_deg)
ip.write(0x10C, float_to_sfix16_en12(theta_rad))

# Arrays to store our plotting data
angles_to_test = np.linspace(-90, 90, 181) # Test every degree from -90 to 90
magnitudes = []

print("Sweeping angles and querying the FPGA...")
```

```

for angle in angles_to_test:
    # 1. Calculate the phase shift for this specific arrival angle
    arrival_rad = math.radians(angle)
    phase_shift = math.pi * math.sin(arrival_rad)

    # 2. Generate the 4 incoming signals (Amplitude = 0.5)
    amplitude = 0.5
    signals = []
    for i in range(4):
        element_phase = -(i * phase_shift)
        real_part = amplitude * math.cos(element_phase)
        imag_part = amplitude * math.sin(element_phase)
        signals.append((real_part, imag_part))

    # 3. Send to hardware
    write_complex_input(0x100, signals[0][0], signals[0][1], 0x108) # In1
    write_complex_input(0x110, signals[1][0], signals[1][1], 0x118) # In2
    write_complex_input(0x120, signals[2][0], signals[2][1], 0x128) # In3
    write_complex_input(0x130, signals[3][0], signals[3][1], 0x138) # In4

    # 4. Read the output from the FPGA
    y_real, y_imag = read_complex_output()
    mag = math.sqrt(y_real**2 + y_imag**2)
    magnitudes.append(mag)

# Plotting the results
plt.figure(figsize=(10, 6))
plt.plot(angles_to_test, magnitudes, linewidth=2, color='b')
plt.title(f"Hardware Beamformer Radiation Pattern (Steered to {steer_angle_deg}°)")
plt.xlabel("Angle of Arrival (Degrees)")
plt.ylabel("Output Magnitude")

```

```
plt.axvline(x=steer_angle_deg, color='r', linestyle='--', label=f'Steering Direction
({steer_angle_deg}°)')
plt.grid(True)
plt.legend()
plt.xlim(-90, 90)
plt.show()
```

Spatial Sweep and Radiation Pattern Generation

This subsequent script utilizes the initialized hardware to perform a continuous sweep of incoming wavefronts from -90° to $+90^\circ$. It loops through the hardware execution for each angle, stores the resulting magnitude, and uses matplotlib to plot the spatial directivity of the beamformer.

```
from pynq import Overlay, MMIO
import matplotlib.pyplot as plt
import numpy as np
import math

# =====
# 1. Initialization and Configuration
# =====

print("Loading overlay...")
overlay = Overlay("beamform_ver_1_wrapper.bit")
ip = MMIO(0x40000000, 0x1000)

# =====
# 2. Hardware Helper Functions
# =====

def float_to_sfix16_en12(val):
    max_val, min_val = (2**15 - 1) / 4096.0, -(2**15) / 4096.0
    val = max(min(val, max_val), min_val)
    int_val = int(round(val * 4096))
    if int_val < 0: int_val = (1 << 16) + int_val
    return int_val & 0xFFFF
```

```

def sfix16_en12_to_float(val):
    val = val & 0xFFFF
    if val >= 32768: val -= 65536
    return val / 4096.0

def write_complex_input(base_addr, real_val, imag_val, strobe_addr):
    ip.write(base_addr, float_to_sfix16_en12(real_val))
    ip.write(base_addr + 0x04, float_to_sfix16_en12(imag_val))
    ip.write(strobe_addr, 1)

def read_complex_output():
    ip.write(0x148, 1) # Output strobe
    return sfix16_en12_to_float(ip.read(0x140)), sfix16_en12_to_float(ip.read(0x144))

# =====
# 3. Time-Domain Sine Wave Execution
# =====
ip.write(0x004, 1) # Enable IP

# Steer the FPGA to 30 degrees
steer_angle_deg = 30.0
ip.write(0x10C, float_to_sfix16_en12(math.radians(steer_angle_deg)))

# Generate time steps for 2 full cycles of a sine wave
time_steps = np.linspace(0, 4 * math.pi, 150)
phase_shift = math.pi * math.sin(math.radians(steer_angle_deg))

# Lists to hold plotting data
in1_plot, in2_plot, in3_plot, in4_plot, out_plot = [], [], [], [], []

print("Running continuous sine wave through the FPGA fabric...")

for t in time_steps:

```

```

# Calculate the instantaneous amplitude of the sine wave
# We apply the physical delay (phase shift) to each element
sig0_r = 0.5 * math.cos(t - 0 * phase_shift)
sig0_i = 0.5 * math.sin(t - 0 * phase_shift)

sig1_r = 0.5 * math.cos(t - 1 * phase_shift)
sig1_i = 0.5 * math.sin(t - 1 * phase_shift)

sig2_r = 0.5 * math.cos(t - 2 * phase_shift)
sig2_i = 0.5 * math.sin(t - 2 * phase_shift)

sig3_r = 0.5 * math.cos(t - 3 * phase_shift)
sig3_i = 0.5 * math.sin(t - 3 * phase_shift)

# Push to hardware
write_complex_input(0x100, sig0_r, sig0_i, 0x108)
write_complex_input(0x110, sig1_r, sig1_i, 0x118)
write_complex_input(0x120, sig2_r, sig2_i, 0x128)
write_complex_input(0x130, sig3_r, sig3_i, 0x138)

# Read the real-time computed output from the fabric
y_real, y_imag = read_complex_output()

# Store the REAL parts to draw standard sine waves on the graph
in1_plot.append(sig0_r)
in2_plot.append(sig1_r)
in3_plot.append(sig2_r)
in4_plot.append(sig3_r)
out_plot.append(y_real)

# =====
# 4. Draw the Sine Wave Plot
# =====
plt.figure(figsize=(12, 6))

```

```
# Plot the 4 staggered input waves (dashed lines)
plt.plot(time_steps, in1_plot, linestyle='--', alpha=0.7, label='Antenna 1 Input')
plt.plot(time_steps, in2_plot, linestyle='--', alpha=0.7, label='Antenna 2 Input')
plt.plot(time_steps, in3_plot, linestyle='--', alpha=0.7, label='Antenna 3 Input')
plt.plot(time_steps, in4_plot, linestyle='--', alpha=0.7, label='Antenna 4 Input')

# Plot the massive output wave from the FPGA (solid bold line)
plt.plot(time_steps, out_plot, color='black', linewidth=3, label='Combined FPGA Output')

plt.title("Time-Domain Sine Waves (Hardware Output)", fontsize=14, fontweight='bold')
plt.xlabel("Time", fontsize=12)
plt.ylabel("Signal Amplitude", fontsize=12)
plt.grid(True, alpha=0.5)
plt.legend(loc='upper right')
plt.show()
```